

Processando textos com spaCy

Trabalhando com Texto em Python I · Unidade 2

CiberExt 26-29 · FEELT38103 · Universidade Federal de Uberlândia

2026

Sumário

1. Primeiros passos	1
2. Realizando tarefas básicas de PLN com spaCy	2
Quiz	8
Resumo da unidade	9

Objetivos de aprendizagem

Ao final desta unidade, você deverá:

- conhecer alguns dos **conceitos e tarefas fundamentais** do processamento de linguagem natural (PLN);
- saber **realizar tarefas simples** de PLN usando a biblioteca **spaCy**.

1. Primeiros passos

Para começar, importamos o **spaCy**, uma das muitas bibliotecas disponíveis para processamento de linguagem natural em Python.

```
# Importa a biblioteca spaCy
import spacy
```

Dica prática: a primeira importação do spaCy pode demorar alguns segundos, porque a biblioteca carrega dependências pesadas (redes neurais, tokenizadores etc.). Isso é normal.

Para realizar tarefas de processamento de linguagem natural em um idioma específico, precisamos carregar um **modelo de linguagem** treinado para executar essas tarefas naquele idioma.

O spaCy oferece suporte a muitos idiomas, mas disponibiliza **modelos pré-treinados** apenas para um subconjunto deles. Esses modelos vêm em tamanhos e “sabores” diferentes — exploraremos essas diferenças mais adiante no curso. Para nos familiarizarmos com as tarefas básicas de PLN, começaremos com um modelo pequeno para o **inglês**.

Os modelos de linguagem são carregados com a função `load()` do spaCy, que recebe o **nome do modelo** como entrada:

```
# Carrega o modelo pequeno de lingua inglesa e o atribui a variavel 'nlp'
nlp = spacy.load('en_core_web_sm')

# Chama a variavel para examinar o objeto
nlp
```

Chamar a variável `nlp` retorna um objeto `Language` do spaCy, que contém o modelo de linguagem para o inglês. Em essência, o objeto `Language` do spaCy é um **pipeline** (uma sequência de etapas de processamento) que usa o modelo de linguagem para executar diversas tarefas de PLN — tarefas que veremos a seguir.

1.1 O que é um modelo de linguagem?

A maioria dos modelos de linguagem modernos é baseada em **estatística**, e não em regras definidas manualmente por humanos. Modelos de linguagem estatísticos se apoiam em **probabilidades**, respondendo perguntas como:

- Qual a probabilidade de uma determinada frase ocorrer em um idioma?
- Qual a probabilidade de uma determinada palavra ocorrer em uma sequência de palavras?

Considere as frases a seguir, extraídas dos artigos de jornal usados na unidade anterior:

*“From financial exchanges in **OCULTO** Manhattan to cloakrooms in Washington and homeless shelters in California, unfamiliar rituals were the order of the day.”*

*“Security precautions were being taken around the **OCULTO** as the deadline for Iraq to withdraw from Kuwait neared.”*

Você provavelmente consegue **arriscar um bom palpite** sobre as palavras ocultas, com base no seu conhecimento da língua inglesa e do mundo em geral (no primeiro caso, algo como *“lower”*; no segundo, *“country”*).

Da mesma forma, construir um modelo de linguagem estatístico envolve observar a ocorrência de palavras em **grandes corpora** e calcular suas probabilidades de ocorrência em um determinado contexto. O modelo é então treinado fazendo previsões e ajustando-se com base nos erros cometidos durante essas previsões.

1.2 Como os modelos de linguagem são treinados?

O modelo pequeno para o inglês, por exemplo, é treinado em um corpus chamado **OntoNotes 5.0**, que reúne textos de gêneros variados — notícias de agências de imprensa, notícias de telejornal, conversas telefônicas e transmitidas, e blogs. Isso permite que o corpus cubra a variação linguística tanto da língua **escrita** quanto **falada** em inglês.

O corpus OntoNotes 5.0 é composto por mais do que texto puro: suas anotações incluem **classes gramaticais** (*part-of-speech tags*), **dependências sintáticas** e **correferências** entre palavras. Isso permite modelar não apenas a ocorrência de palavras ou sequências de palavras, mas também suas **características gramaticais**.

2. Realizando tarefas básicas de PLN com spaCy

Para processar um texto usando o objeto `Language` que contém o modelo de linguagem para o inglês, basta **chamar** o objeto `nlp` sobre algum texto.

Vamos definir uma frase de teste simples — um objeto *string* do Python, guardado na variável `text`:

```
# Atribui uma frase de exemplo a variavel 'text'
text = "The Federal Bureau of Investigation has been ordered to track down as many as
→ 3,000 Iraqis in this country whose visas have expired, the Justice Department said
→ yesterday."

# Chama a variavel para examinar o resultado
text
```

Passar a variável `text` para o objeto `Language nlp` retorna um objeto `Doc` do spaCy (abreviação de *document*, “documento”). Em processamento de linguagem natural, trechos maiores de texto costumam ser chamados de “documentos”, ainda que, neste caso, nosso documento consista em uma única frase.

Esse objeto contém tanto o texto de entrada (`text`) quanto os **resultados** do processamento de linguagem natural feito pelo spaCy.

```
# Alimenta a string em 'text' ao objeto Language em 'nlp'
# Guarda o resultado na variavel 'doc'
doc = nlp(text)
```

O objeto `Doc` agora está guardado na variável `doc`:

```
# Chama a variavel para examinar o objeto
doc
```

Embora a saída se pareça com uma *string* comum do Python, o objeto `Doc` contém uma **riqueza de informações** sobre a estrutura linguística do texto, geradas pelo spaCy ao processar a frase através do seu *pipeline* de PLN. Vamos agora examinar, uma a uma, as tarefas realizadas “nos bastidores”.

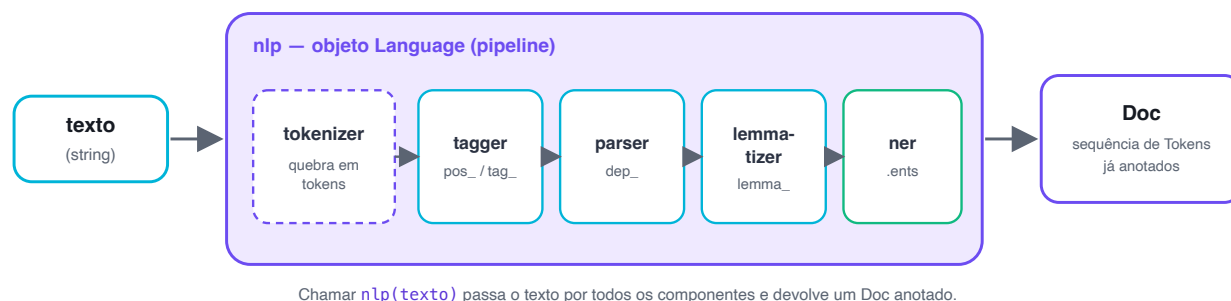


Figura 1: O *pipeline* de PLN do spaCy: chamar `nlp(texto)` passa o texto por vários componentes (tokenizer, tagger, parser, lematizador, NER) e devolve um objeto `Doc` já anotado.

2.1 Tokenização

A primeira tarefa realizada é conhecida como **tokenização**: ela quebra o texto em unidades analíticas que serão processadas posteriormente.

Na maioria dos casos, um **token** corresponde a uma palavra separada por espaço em branco, mas sinais de pontuação também são considerados tokens independentes. Como os computadores tratam palavras como seqüências de caracteres, atribuir aos sinais de pontuação seus próprios tokens evita que a pontuação final “grude” nas palavras que a precedem.

Um objeto `Doc` do spaCy é composto por uma sequência de objetos `Token`, que armazenam os resultados das diversas tarefas de PLN. Vamos imprimir cada objeto `Token` guardado em `doc`:

```
# Percorre os itens no objeto Doc, usando a variavel 'token' para se referir aos itens da
→ lista
for token in doc:

    # Imprime cada token
    print(token)
```

A saída mostra um `Token` por linha. Como esperado, sinais de pontuação como `.` e `,` constituem seus próprios `Tokens`.

2.2 Anotação morfossintática (*part-of-speech tagging*)

A **anotação morfossintática** (*POS tagging*) é a tarefa de determinar a **classe gramatical** de um token. Isso é crucial para desambiguação, já que classes gramaticais diferentes podem ter formas idênticas.

Considere o exemplo em inglês: “*The sailor **dogs** the hatch.*” (algo como “O marinheiro tranca a escotilha”). O presente do verbo *to dog* (prender/trancar algo com algo) é exatamente igual ao plural do substantivo *dog* (cachorro): **dogs**. Para identificar a classe correta, é preciso examinar o **contexto** em que a palavra aparece.

O spaCy fornece dois tipos de anotação morfossintática, uma **genérica** (*coarse*) e uma **detalhada** (*fine-grained*), guardadas respectivamente nos atributos `pos_` e `tag_`. Acessamos os atributos de um objeto Python inserindo o atributo após o objeto, separados por um ponto final — por exemplo, `token.pos_`.

```
# Percorre os itens no objeto Doc, usando a variavel 'token' para se referir aos itens da
→ lista
for token in doc:

    # Imprime o token e as anotacoes morfossintaticas genericas e detalhadas
    print(token, token.pos_, token.tag_)
```

As anotações genéricas disponíveis em `pos_` são baseadas no conjunto de rótulos do projeto **Universal Dependencies**. Já as anotações detalhadas em `tag_` são baseadas no corpus OntoNotes 5.0 apresentado acima. Diferentemente das anotações genéricas, as detalhadas também codificam informação gramatical adicional — as anotações de verbos, por exemplo, se distinguem por **aspecto** e **tempo verbal**.

2.3 Análise morfológica

Morfemas são as menores unidades gramaticais que carregam significado. Reconhecem-se geralmente dois tipos: **morfemas livres**, que consistem em palavras capazes de existir sozinhas, e **morfemas presos**, que flexionam outros morfemas. No inglês, morfemas presos incluem sufixos como `-s`, usado para indicar o plural de um substantivo.

Em outras palavras, os morfemas moldam a forma externa de uma palavra, e essas formas estão associadas a funções gramaticais específicas.

O spaCy realiza a análise morfológica automaticamente, guardando o resultado no atributo `morph` de um objeto `Token`:

```
# Percorre os itens no objeto Doc, usando a variavel 'token' para se referir aos itens da
→ lista
for token in doc:
```

```
# Imprime o token e o resultado da analise morfologica
print(token, token.morph)
```

Como mostra a saída, **nem todos os tokens** têm informação morfológica — alguns consistem em morfemas livres, sem flexão. Para recuperar informação morfológica de um **Token**, usamos o método `get()` do atributo `morph`. Podemos usar colchetes `[]` para acessar itens no objeto **Doc**. A linha a seguir recupera a informação morfológica de **aspecto** para o 23º token do **Doc** (índice 22):

```
# Recupera informacao morfologica sobre aspecto para o Token no indice 22 do objeto Doc
doc[22].morph.get('Aspect')
```

Isso retorna uma lista com um único item, a *string* `Perf`, que se refere ao **aspecto perfectivo**.

O que acontece se tentarmos recuperar uma característica morfológica que o token **não possui**? Vamos tentar recuperar a informação de aspecto para o 22º token (índice 21):

```
# Recupera informacao morfologica sobre aspecto para o Token no indice 21 do objeto Doc
doc[21].morph.get('Aspect')
```

Isso retorna uma **lista vazia**, indicada pelos colchetes `[]` sem nada entre eles.

Para recuperar **toda** a informação morfológica disponível para um dado **Token**, a melhor solução é usar o método `to_dict()` do atributo `morph`. Isso retorna um **dicionário**, uma estrutura de dados do Python composta por pares de chave e valor:

```
# Recupera informacao morfologica para o Token no indice 21 do objeto Doc
# Usa o metodo to_dict() para converter o resultado em um dicionario
doc[21].morph.to_dict()
```

Um dicionário Python é delimitado por chaves `{ }`. Cada par chave/valor é separado por dois-pontos `:`. Nesse caso, tanto as chaves quanto os valores são objetos *string*. O valor guardado em uma chave pode ser acessado colocando o nome da chave entre colchetes `[]` logo após o nome do dicionário:

```
# Atribui a informacao morfologica ao dicionario 'morph_dict'
morph_dict = doc[21].morph.to_dict()

# Recupera o valor correspondente a chave 'Mood'
morph_dict['Mood']
```

Dicionários são uma estrutura de dados poderosa em Python, que usaremos com frequência para guardar informações.

2.4 Análise sintática (*parsing* de dependências)

A **análise sintática** (ou *parsing* de dependências) é a tarefa de definir as **dependências sintáticas** entre tokens. Essas dependências ficam disponíveis no atributo `dep_` de um objeto **Token**:

```
# Percorre os itens no objeto Doc, usando a variavel 'token' para se referir aos itens da
→ lista
for token in doc:

    # Imprime o token e sua etiqueta de dependencia
    print(token, token.dep_)
```

Diferentemente das anotações morfofossintáticas, associadas a um único **Token**, as etiquetas de dependência indicam uma **relação** entre dois tokens. Para entender melhor essas relações sintáticas, vamos usar alguns atributos adicionais de cada **Token**:

- **i**: a posição do token no **Doc**;
- **token**: o próprio token;
- **dep_**: a etiqueta da relação sintática;
- **head** e **i**: o token que **governa** o token atual, e o índice desse token-governante.

Isso ilustra como os atributos do Python podem ser combinados de forma flexível: o atributo **head** aponta para outro **Token**, que por sua vez tem o atributo **i** com seu próprio índice no **Doc**. Podemos combinar os dois atributos e acessar essa informação com **.head.i**:

```
# Percorre os itens no objeto Doc, usando a variavel 'token' para se referir aos itens da
↳ lista
for token in doc:

    # Imprime o indice do token atual, o token, a dependencia, o governante e seu indice
    print(token.i, token, token.dep_, token.head.i, token.head)
```

Embora a saída acima ajude a esclarecer as dependências sintáticas entre os tokens, elas costumam ser **muito mais fáceis de perceber** por meio de diagramas. O spaCy oferece uma ferramenta de visualização chamada **displaCy**, importável com o comando:

```
from spacy import displacy
```

O módulo **displacy** tem uma função chamada **render()**, que recebe um objeto **Doc** como entrada. Para desenhar uma árvore de dependências, passamos o **Doc doc** à função **render()** com dois argumentos:

- **style**: o valor **'dep'** instrui o displaCy a desenhar uma visualização de dependências sintáticas;
- **options**: recebe um dicionário Python como entrada. Passamos um dicionário com a chave **compact** e o valor booleano **True** para instruir o displaCy a desenhar uma árvore compacta.

```
displacy.render(doc, style='dep', options={'compact': True})
```

As dependências sintáticas são visualizadas por meio de **linhas** que partem do token-governante em direção ao token governado por ele. As etiquetas de dependência são baseadas no projeto **Universal Dependencies**, um arcabouço para descrever características morfológicas e sintáticas entre diferentes idiomas.

Se você não souber o significado de uma etiqueta específica, o spaCy oferece uma função para explicá-las, **explain()**, que recebe a etiqueta como entrada (atenção: as etiquetas diferenciam maiúsculas de minúsculas):

```
spacy.explain('pobj')
```

Por fim, se você está se perguntando sobre os sublinhados **_** nos nomes dos atributos: o spaCy codifica todas as **strings** mapeando-as para **valores de hash** (uma representação numérica) por eficiência computacional. Vamos imprimir o primeiro token do **Doc** (**doc[0]**) e suas dependências para examinar como isso funciona:

```
print(doc[0], doc[0].dep, doc[0].dep_)
```

Como se vê, o valor de hash **415** está reservado para a etiqueta correspondente a um determinante (**det**).

Atenção: se você quiser uma saída legível para humanos na análise de dependências e o spaCy devolver sequências de números, provavelmente você esqueceu de adicionar o sublinhado ao nome do atributo (por exemplo, usou `dep` em vez de `dep_`).

2.5 Segmentação de sentenças

O spaCy também segmenta objetos `Doc` em **sentenças** — tarefa conhecida como **segmentação de sentenças**. Essa segmentação impõe estrutura adicional a textos maiores: ao determinar os limites de uma sentença, podemos restringir tarefas como a análise de dependências a sentenças individuais.

O spaCy disponibiliza o resultado da segmentação de sentenças no atributo `sents` de um objeto `Doc`. Vamos percorrer as sentenças contidas em `doc` e contá-las usando a função `enumerate()` do Python, que retorna uma contagem crescente a cada item do laço:

```
# Percorre as sentencas no objeto Doc e as conta usando enumerate()
for number, sent in enumerate(doc.sents):

    # Imprime o numero e a sentenca
    print(number, sent)
```

Isso retorna apenas **uma sentença**, mas o objeto `Doc` poderia facilmente conter um texto mais longo com múltiplas sentenças, como uma reportagem inteira.

2.6 Lematização

Um **lema** é a forma base de uma palavra. Tenha em mente que, a menos que instruído explicitamente, o computador não sabe distinguir formas singular e plural de uma palavra — ele as trata como tokens distintos, porque suas formas diferem.

Se quisermos contar a **ocorrência de palavras**, por exemplo, é necessário um processo chamado **lematização**, que agrupa as diferentes formas de um mesmo token. Os lemas ficam disponíveis para cada `Token` no atributo `lemma_`:

```
# Percorre os itens no objeto Doc, usando a variavel 'token' para se referir aos itens da
→ lista
for token in doc:

    # Imprime o token e seu lema
    print(token, token.lemma_)
```

2.7 Reconhecimento de entidades nomeadas (*named entity recognition*, NER)

O **reconhecimento de entidades nomeadas** (NER) é a tarefa de identificar e classificar entidades mencionadas em um texto. O spaCy reconhece as entidades nomeadas anotadas no corpus OntoNotes 5 — pessoas, locais geográficos e produtos, entre outros exemplos.

Usamos o atributo `.ents` do objeto `Doc` para obter as entidades nomeadas:

```
doc.ents
```

Isso retorna uma **tupla** com as entidades nomeadas. Cada item da tupla é um objeto `Span` do spaCy — objetos `Span` podem conter **múltiplos** objetos `Token`, já que muitas entidades nomeadas se estendem por

mais de um token. As entidades nomeadas e seus tipos ficam guardados nos atributos `.text` e `.label_` de cada `Span`. Vamos percorrer a tupla e imprimir ambos os atributos:

```
# Percorre as entidades nomeadas no objeto Doc
for ent in doc.ents:

    # Imprime a entidade nomeada e sua etiqueta
    print(ent.text, ent.label_)
```

Como se vê, a maioria das entidades nomeadas identificadas consiste em **múltiplos tokens**, por isso são representadas como objetos `Span`. Podemos confirmar isso acessando a primeira entidade nomeada (índice 0, já que o Python conta do zero) e passando o objeto à função `type()`:

```
# Verifica o tipo do objeto usado para armazenar entidades nomeadas
type(doc.ents[0])
```

Objetos `Span` do spaCy têm vários argumentos úteis. Mais importante: os atributos `start` e `end` retornam os **índices dos tokens** que determinam onde o `Span` começa e termina no `Doc`. Vamos examinar isso com mais detalhe, imprimindo os atributos `start` e `end` da primeira entidade nomeada:

```
# Imprime a entidade nomeada e os índices de seus tokens de início e fim
print(doc.ents[0], doc.ents[0].start, doc.ents[0].end)
```

A entidade nomeada começa no índice 0 e termina no índice 5 do `Doc`. Se recuperarmos o sexto token do `Doc` (índice 5), veremos que ele corresponde ao token "has":

```
doc[5]
```

Isso mostra que o índice retornado por `end` **não** corresponde ao último token do `Span` que contém a entidade nomeada, mas sim ao índice do **primeiro token seguinte** ao `Span`. Vamos examinar isso percorrendo o recorte (*slice*) do `Doc` correspondente à primeira entidade nomeada:

```
# Percorre um recorte do objeto Doc que cobre a primeira entidade nomeada
for token in doc[doc.ents[0].start: doc.ents[0].end]:

    # Imprime o token e seu índice
    print(token, token.i)
```

Como se vê, o atributo `start` indica onde o `Span` **começa**, enquanto `end` indica onde o `Span` **terminou** (ou seja, o token logo após o seu fim).

Também podemos renderizar as entidades nomeadas com o `displacy`, o mesmo módulo usado acima para visualizar as dependências sintáticas. Note que, desta vez, passamos a *string* 'ent' ao argumento `style` para indicar que queremos visualizar **entidades nomeadas**:

```
displacy.render(doc, style='ent')
```

Se você não reconhecer uma etiqueta usada para uma entidade nomeada, pode sempre pedir uma explicação ao spaCy:

```
spacy.explain('NORP')
```


Quiz

Marque a alternativa correta (a resposta certa está destacada com ✓).

1. A tarefa que quebra o texto em unidades menores chama-se:

1. Tokenização ✓
2. Lematização
3. Segmentação de sentenças

2. Qual atributo do token dá a classe gramatical genérica?

1. `pos_` ✓
2. `tag_`
3. `dep_`

3. O lema de uma palavra é:

1. A forma base da palavra ✓
2. Sempre o plural
3. A raiz fonética

4. As entidades nomeadas de um `Doc` ficam em qual atributo?

1. `.ents` ✓
2. `.sents`
3. `.noun_chunks`

5. Por que muitos atributos têm uma versão terminada em sublinhado (ex.: `dep_`)?

1. Ela dá o texto legível, em vez do código numérico (hash) ✓
2. Ela é mais rápida
3. Ela é obrigatória

6. Uma entidade nomeada com vários tokens é representada por um objeto:

1. `Span` ✓
2. `Token`
3. `Doc`

Resumo da unidade

Nesta unidade, você aprendeu a:

1. **carregar** um modelo de linguagem com `spacy.load()` e entender o que é e como é treinado um modelo estatístico de linguagem;
2. **processar** um texto com o objeto `Language(nlp(text))`, obtendo um objeto `Doc`;
3. **tokenizar** o texto e percorrer os objetos `Token` resultantes;
4. **anotar classes gramaticais** (`pos_`, `tag_`) e realizar **análise morfológica** (`morph`, `to_dict()`);
5. **analisar dependências sintáticas** (`dep_`, `head`), visualizando-as com `displacy.render(..., style='dep')`;
6. **segmentar sentenças** (`doc.sents`) e **lematizar** tokens (`lemma_`);
7. **reconhecer entidades nomeadas** (`doc.ents`, `.text`, `.label_`), entendendo os atributos `start/end` de um `Span`, e visualizá-las com `displacy.render(..., style='ent')`.

Exercícios sugeridos

1. Processe uma das notícias completas da Unidade 1 com `nlp()` e conte quantas sentenças o `Doc` resultante contém.
 2. Liste todos os tokens classificados como verbo (`token.pos_ == 'VERB'`) em um texto à sua escolha.
 3. Extraia todas as entidades nomeadas do tipo pessoa (`PERSON`) de um texto e imprima seus lemas.
 4. Use `spacy.explain()` para descobrir o significado de pelo menos três etiquetas de dependência (`dep_`) diferentes das usadas nos exemplos acima.
-

Referências

- Rognvaldsson, E. et al. *Applied Language Technology* (MOOC). Universidade de Helsinque. <https://applied-language-technology.mooc.fi/>
- de Marneffe, M.-C. et al. (2021). Universal Dependencies. *Computational Linguistics*.
- Honnibal, M.; Montani, I. et al. *spaCy: Industrial-strength Natural Language Processing in Python*. <https://spacy.io/>